

NumPy Exercises — Simple Coding Tasks

Creating NumPy Arrays — 10 Simple Tasks

1. Create `a = np.array([0,1,2,3,4])` and print `a` .
2. Make `b = np.zeros((2,3))` and print `b` .
3. Make `c = np.ones((2,2))` and multiply by 5; print `c` .
4. Create `d = np.arange(0,10)` and print the last element using negative indexing.
5. Use `np.linspace(0,1,5)` and print the result.
6. Create `e = np.full((2,3), 9)` and print `e` .
7. Create a 3×3 identity matrix with `np.eye(3)` and print it.
8. Make a float array `f = np.array([1.2, 2.5, 3.7])` , convert to int with `astype` , and print.
9. Create `g = np.arange(6)` and reshape to `(2,3)` ; print `g` .
10. Create `h = np.arange(0,20,2)` and print its `dtype` and `shape` .

Indexing and Slicing — 10 Simple Tasks

1. Create `arr = np.arange(12).reshape(4,3)` and print `arr` .
2. Flatten `arr` with `arr.flatten()` and print the flattened array.
3. From the flattened array, print elements `3:6` .
4. Make `b = arr[1]` and print the second row.
5. Use negative indexing to print the last row (`arr[-1]`).
6. Create `sel = arr[[0,2]]` to pick rows 0 and 2 and print `sel` .
7. Create a 1D array and show boolean indexing: `x = np.array([1,2,3,4]); print(x[x>2])` .
8. Use `np.where` to find indices where `x` is even.
9. Copy a slice: `s = arr.flatten()[2:5].copy()` ; change `s[0]` and show original unchanged.
10. Show slicing with step: `print(np.arange(10)[::2])` .

Multidimensional Indexing and Axis — 10 Simple Tasks

1. Create `arr3D = np.arange(24).reshape(2,3,4)` and print `arr3D.shape` .
2. Print `np.sum(arr3D, axis=0)` and its shape.
3. Print `np.sum(arr3D, axis=1)` and its shape.
4. Set `arr3D[:,0,:] = 0` and print `arr3D` .

5. Create `a2 = np.array([[1,2,3],[4,5,6]])` and add `[10,10,10]` to each row using broadcasting; print result.
6. Show `a2.T` (transpose) and print shapes before and after.
7. Use `keepdims=True`: `print(np.mean(a2, axis=1, keepdims=True).shape)`.
8. Demonstrate assigning a compatible slice value: `arr3D[:,1,:] = np.ones((2,4))*5` and print `arr3D`.
9. Reshape a 1D `ts = np.arange(20)` into `(5,4)` and compute per-row mean.
10. Given `data = np.arange(12).reshape(3,4)`, standardize each column (feature) by subtracting column mean and dividing by column std; print result.